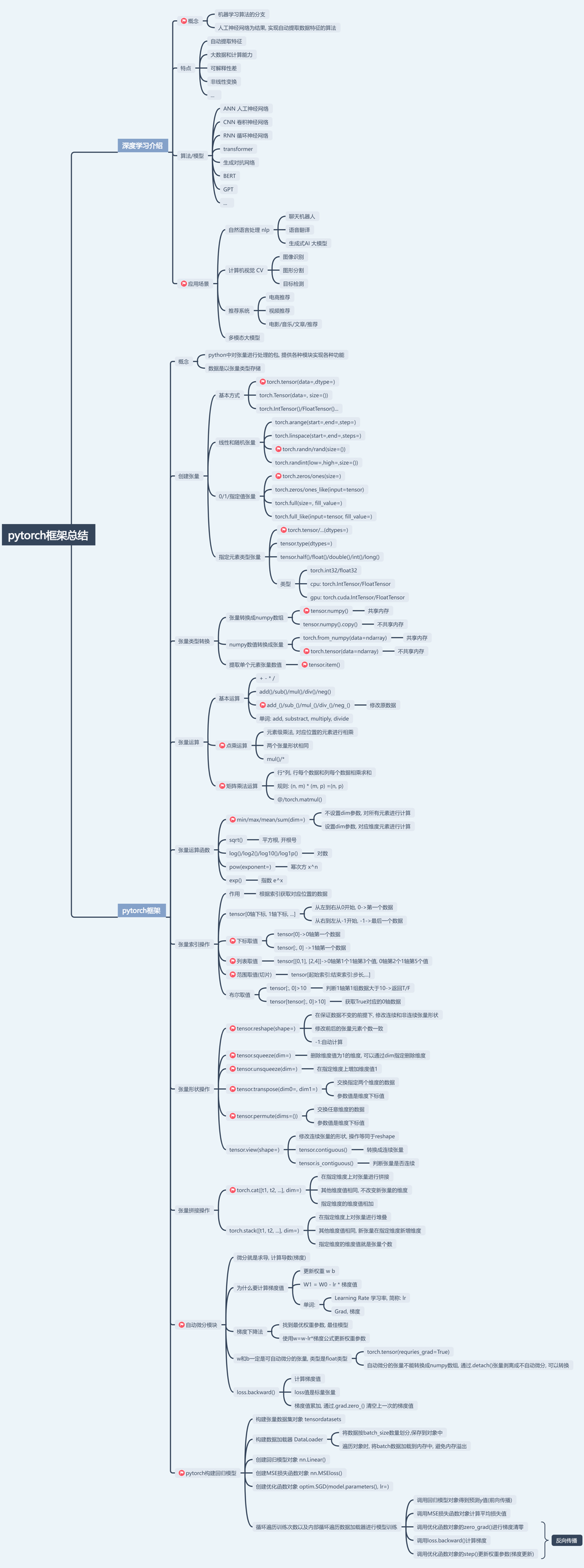
pytorch框架总结
深度学习介绍
概念
机器学习算法的分支
人工神经网络为结果, 实现自动提取数据特征的算法
特点
自动提取特征
大数据和计算能力
可解释性差
非线性变换
...
算法/模型
ANN 人工神经网络
CNN 卷积神经网络
RNN 循环神经网络
transformer
生成对抗网络
BERT
GPT
...
应用场景
自然语言处理 nlp
聊天机器人
语音翻译
生成式AI 大模型
计算机视觉 CV
图像识别
图形分割
目标检测
推荐系统
电商推荐
视频推荐
电影/音乐/文章/推荐
多模态大模型
pytorch框架
概念
python中对张量进行处理的包, 提供各种模块实现各种功能
数据是以张量类型存储
创建张量
基本方式
torch.tensor(data=,dtype=)
torch.Tensor(data=, size=())
torch.IntTensor()/FloatTensor()...
线性和随机张量
torch.arange(start=,end=,step=)
torch.linspace(start=,end=,steps=)
torch.randn/rand(size=())
torch.randint(low=,high=,size=())
0/1/指定值张量
torch.zeros/ones(size=)
torch.zeros/ones_like(input=tensor)
torch.full(size=, fill_value=)
torch.full_like(input=tensor, fill_value=)
指定元素类型张量
torch.tensor/...(dtypes=)
tensor.type(dtypes=)
tensor.half()/float()/double()/int()/long()
类型
torch.int32/float32
cpu: torch.IntTensor/FloatTensor
gpu: torch.cuda.IntTensor/FloatTensor
张量类型转换
张量转换成numpy数组
tensor.numpy()
共享内存
tensor.numpy().copy()
不共享内存
numpy数值转换成张量
torch.from_numpy(data=ndarray)
共享内存
torch.tensor(data=ndarray)
不共享内存
提取单个元素张量数值
tensor.item()
张量运算
基本运算
+ - * /
add()/sub()/mul()/div()/neg()
add_()/sub_()/mul_()/div_()/neg_()
修改原数据
单词: add, substract, multiply, divide
点乘运算
元素级乘法, 对应位置的元素进行相乘
两个张量形状相同
mul()/*
矩阵乘法运算
行*列, 行每个数据和列每个数据相乘求和
规则: (n, m) * (m, p) =(n, p)
@/torch.matmul()
张量运算函数
min/max/mean/sum(dim=)
不设置dim参数, 对所有元素进行计算
设置dim参数, 对应维度元素进行计算
sqrt()
平方根, 开根号
log()/log2()/log10()/log1p()
对数
pow(exponent=)
幂次方 x^n
exp()
指数 e^x
张量索引操作
作用
根据索引获取对应位置的数据
tensor[0轴下标, 1轴下标, ...]
从左到右从0开始, 0->第一个数据
从右到左从-1开始, -1->最后一个数据
下标取值
tensor[0]->0轴第一个数据
tensor[:, 0] ->1轴第一个数据
列表取值
tensor[[0,1], [2,4]]->0轴第1个1轴第3个值, 0轴第2个1轴第5个值
范围取值(切片)
tensor[起始索引:结束索引:步长,...]
布尔取值
tensor[:, 0]>10
判断1轴第1组数据大于10->返回T/F
tensor[tensor[:, 0]>10]
获取True对应的0轴数据
张量形状操作
tensor.reshape(shape=)
在保证数据不变的前提下, 修改连续和非连续张量形状
修改前后的张量元素个数一致
-1:自动计算
tensor.squeeze(dim=)
删除维度值为1的维度, 可以通过dim指定删除维度
tensor.unsqueeze(dim=)
在指定维度上增加维度值1
tensor.transpose(dim0=, dim1=)
交换指定两个维度的数据
参数值是维度下标值
tensor.permute(dims=())
交换任意维度的数据
参数值是维度下标值
tensor.view(shape=)
修改连续张量的形状, 操作等同于reshape
tensor.contiguous()
转换成连续张量
tensor.is_contiguous()
判断张量是否连续
张量拼接操作
torch.cat([t1, t2, ...], dim=)
在指定维度上对张量进行拼接
其他维度值相同, 不改变新张量的维度
指定维度的维度值相加
torch.stack([t1, t2, ...], dim=)
在指定维度上对张量进行堆叠
其他维度值相同, 新张量在指定维度新增维度
指定维度的维度值就是张量个数
自动微分模块
微分就是求导, 计算导数(梯度)
为什么要计算梯度值
更新权重 w b
W1 = W0 - lr * 梯度值
单词:
Learning Rate 学习率, 简称: lr
Grad, 梯度
梯度下降法
找到最优权重参数, 最佳模型
使用w=w-lr*梯度公式更新权重参数
w和b一定是可自动微分的张量, 类型是float类型
torch.tensor(requries_grad=True)
自动微分的张量不能转换成numpy数组, 通过.detach()张量剥离成不自动微分, 可以转换
loss.backward()
计算梯度值
loss值是标量张量
梯度值累加, 通过.grad.zero_() 清空上一次的梯度值
pytorch构建回归模型
构建张量数据集对象 tensordatasets
构建数据加载器 DataLoader
将数据按batch_size数量划分,保存到对象中
遍历对象时, 将batch数据加载到内存中, 避免内存溢出
创建回归模型对象 nn.Linear()
创建MSE损失函数对象 nn.MSEloss()
创建优化函数对象 optim.SGD(model.parameters(), lr=)
循环遍历训练次数以及内部循环遍历数据加载器进行模型训练
调用回归模型对象得到预测y值(前向传播)
调用MSE损失函数对象计算平均损失值
调用优化函数对象的zero_grad()进行梯度清零
调用loss.backward()计算梯度
调用优化函数对象的step()更新权重参数(梯度更新)
反向传播